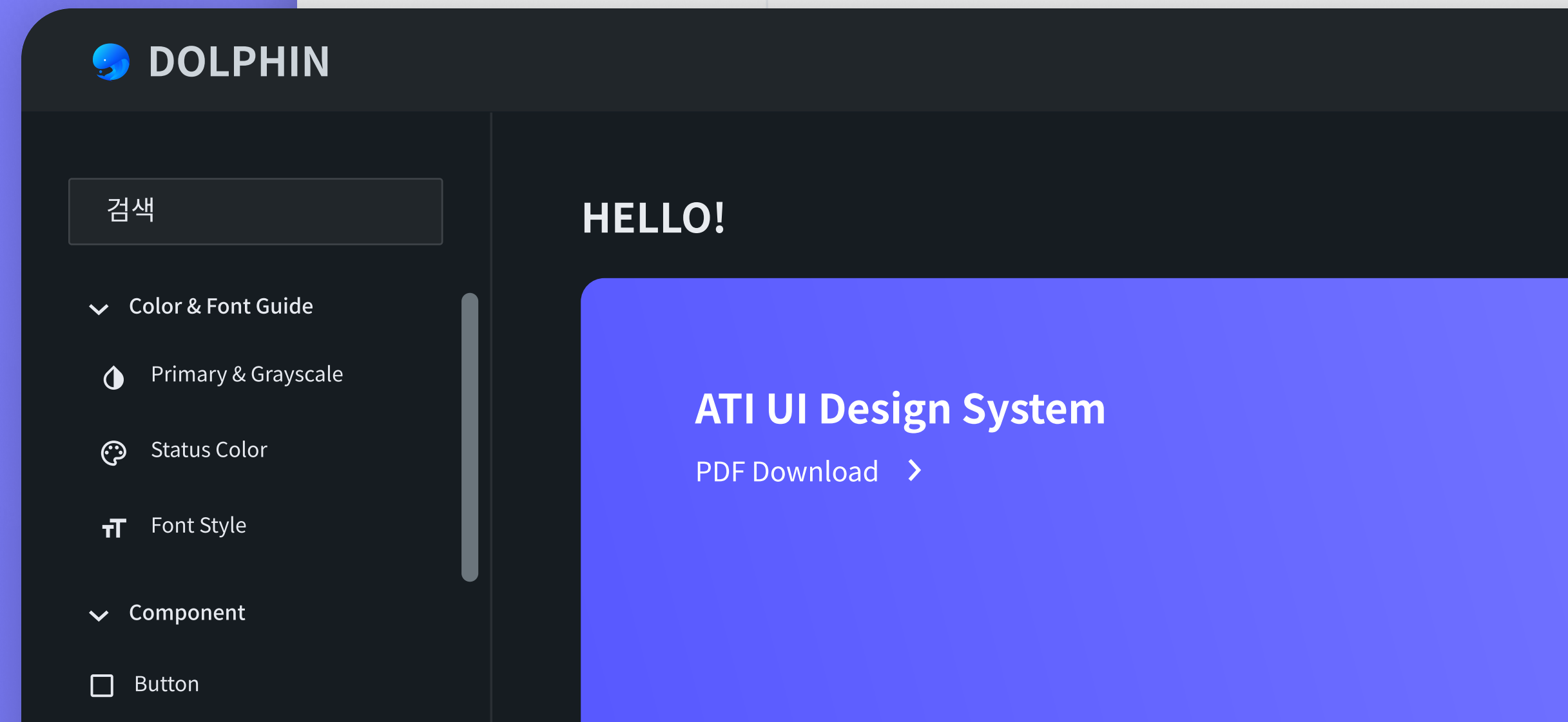
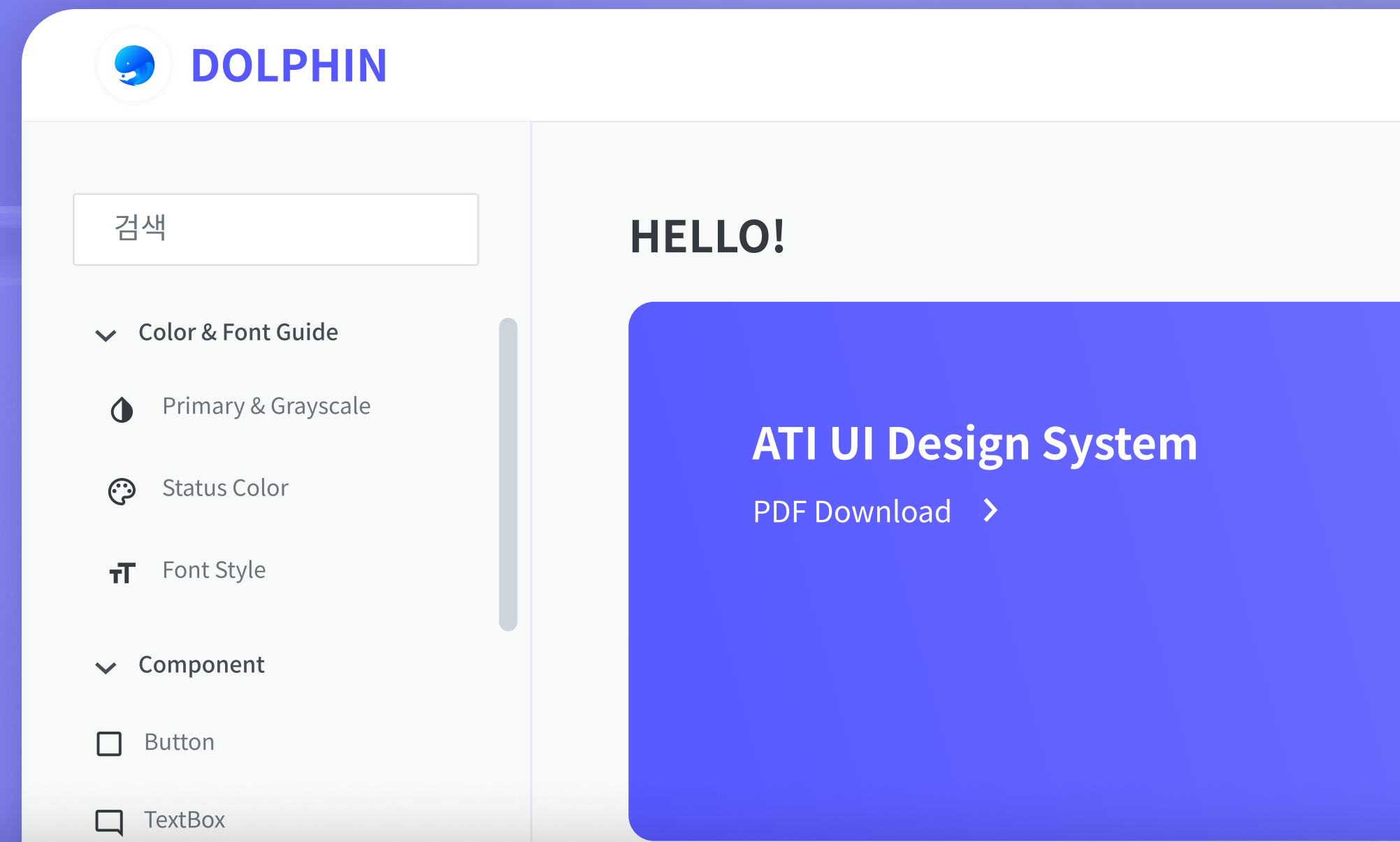




WPF UI 라이브러리 패키지 사용법

Dolphin UI 라이브러리 사용 가이드



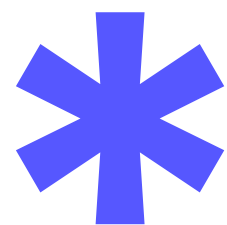
OverView

DOLPHIN 디자인 시스템은 ATI 설비 프로그램의 UI 통일을 위해 설계된 디자인 규칙으로 여러 WPF 프로젝트에서 반복적으로 발생하는 UI 설계와 구현 작업을 줄이고, 제품 전반에 일관된 사용자 경험을 제공하기 위해 시작되었습니다. 설비 프로젝트마다 버튼, 입력창, 팝업, 타이틀바와 같은 UI 요소를 개별적으로 구현할 경우 디자인과 동작 방식이 달라질 수 있으며, 동일한 화면을 반복해서 설계하고 개발해야 하는 비효율이 발생합니다.

DOLPHIN은 이러한 문제를 해결하기 위해 공통으로 사용되는 UI 컴포넌트 요소를 재사용 가능한 Control과 Page Template 형태로 제공하며 컬러, 타이포그래피, 간격, 상태 표현, 테마와 같은 기본 디자인 규칙부터 실제 프로젝트에 적용할 수 있는 화면 구조까지 하나의 기준으로 관리하는 것을 목표로 합니다.

이를 통해 반복적인 UI 설계보다 사용자 경험과 화면 흐름에 집중할 수 있으며, 개발자는 검증된 UI 요소와 구조를 활용하여 보다 빠르고 안정적으로 화면을 구현할 수 있습니다. 이 안내 사항에서는 Dolphin의 구성과 설계 원칙, 주요 기능, 적용 방식을 소개합니다.

INDEX



UI 컴포넌트 라이브러리를
적용하는 과정을
안내합니다

UI 라이브러리 설명 안내

- 디자인 시스템과 UI 라이브러리
- Dolphin UI 라이브러리의 역할
- Dolphin에서 제공하는 Control과 Style
- 개발자가 직접 구현해야 하는 기능 범위

설치 방법 및 업데이트 안내

- Dolphin UI 라이브러리 패키지 설치
- ResourceDictionary 및 Theme 적용
- Dolphin Namespace 등록

View 화면 생성 안내

- UserControl 또는 Window
- View와 ViewModel 파일 구성
- DataContext 연결

Dolphin 컴포넌트 적용

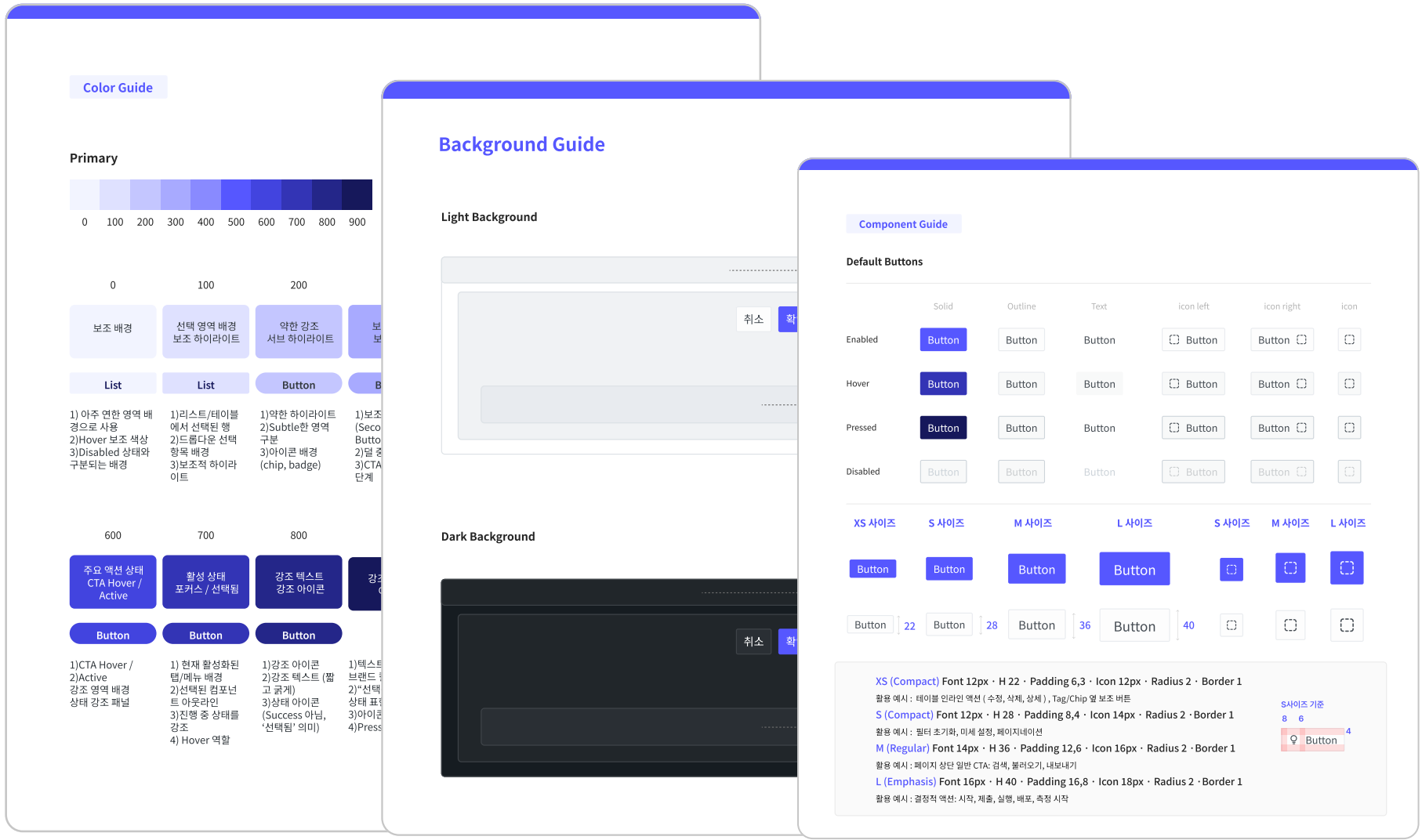
- 기본 WPF Control을 Dolphin Control로 구성
- Button, TextBox 등 적용 예시
- 기본 레이아웃 구성 예시

UI 라이브러리 설명 안내

DOLPHIN 을 사용하기 전에 UI 라이브러리가 무엇인지 안내합니다



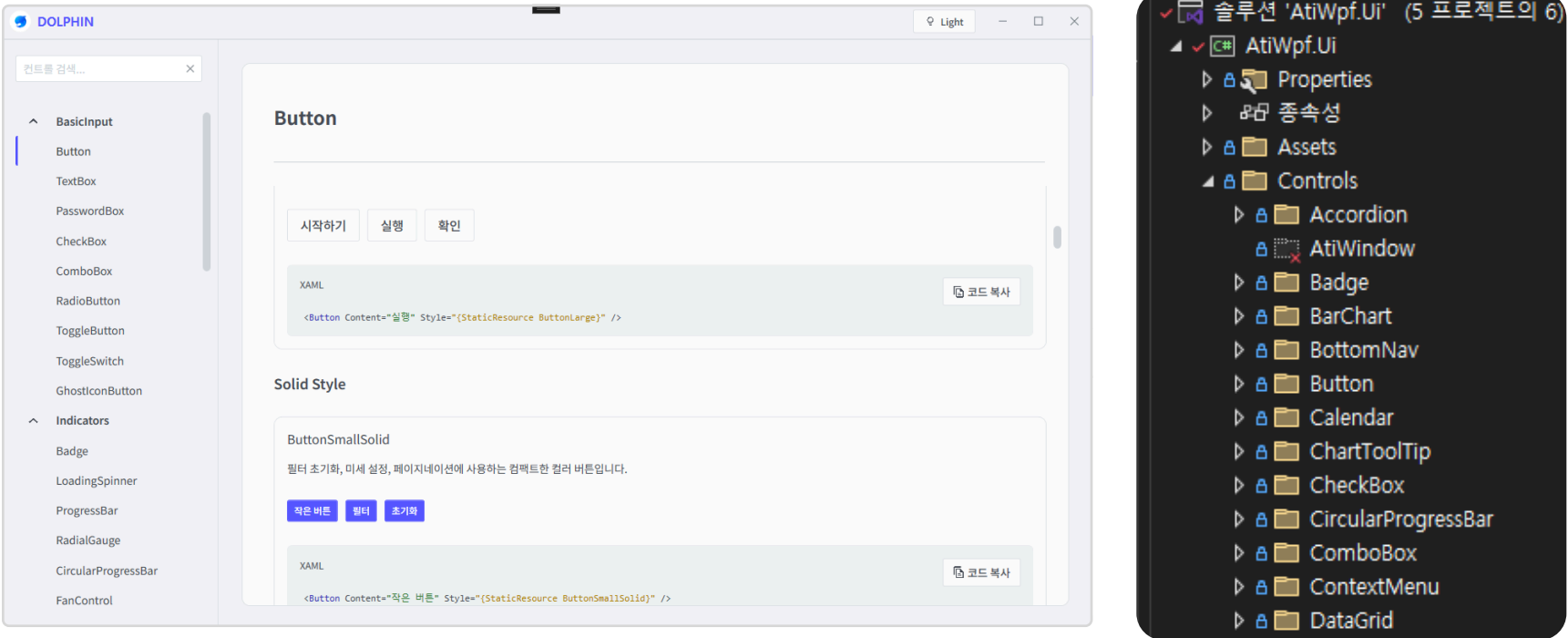
디자인 시스템과 UI 라이브러리



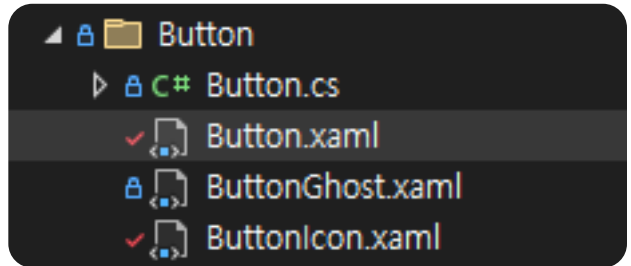
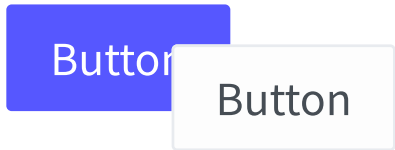
UI 컴포넌트 디자인 시스템 문서

디자인 시스템은

제품의 UI를 일관된 기준으로 설계하기 위한 전체 규칙과 자산을 의미합니다. 색상, 폰트, 간격 등의 기본 디자인 기준뿐만 아니라 컴포넌트의 형태와 상태, 사용 원칙, 화면 구성 방법까지 포함하며, 디자이너와 개발자가 동일한 기준으로 작업할 수 있도록 하는 공통 가이드 역할을 합니다.



코드로 구현한 컴포넌트 Control + Preview 갤러리



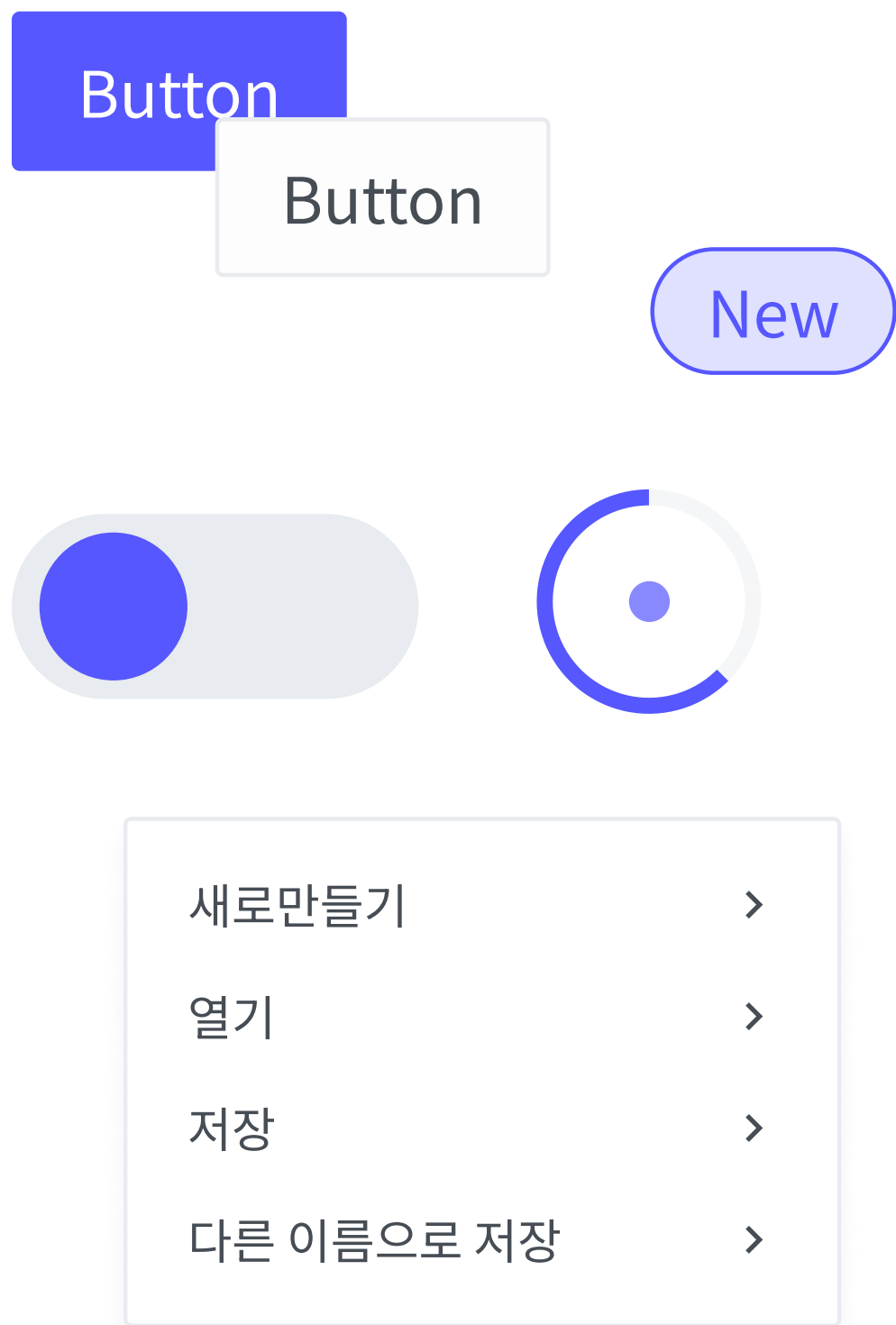
UI 라이브러리는

디자인 시스템에서 정의한 기준을 실제 개발 환경에서 사용할 수 있도록 코드로 구현한 구성 요소입니다.

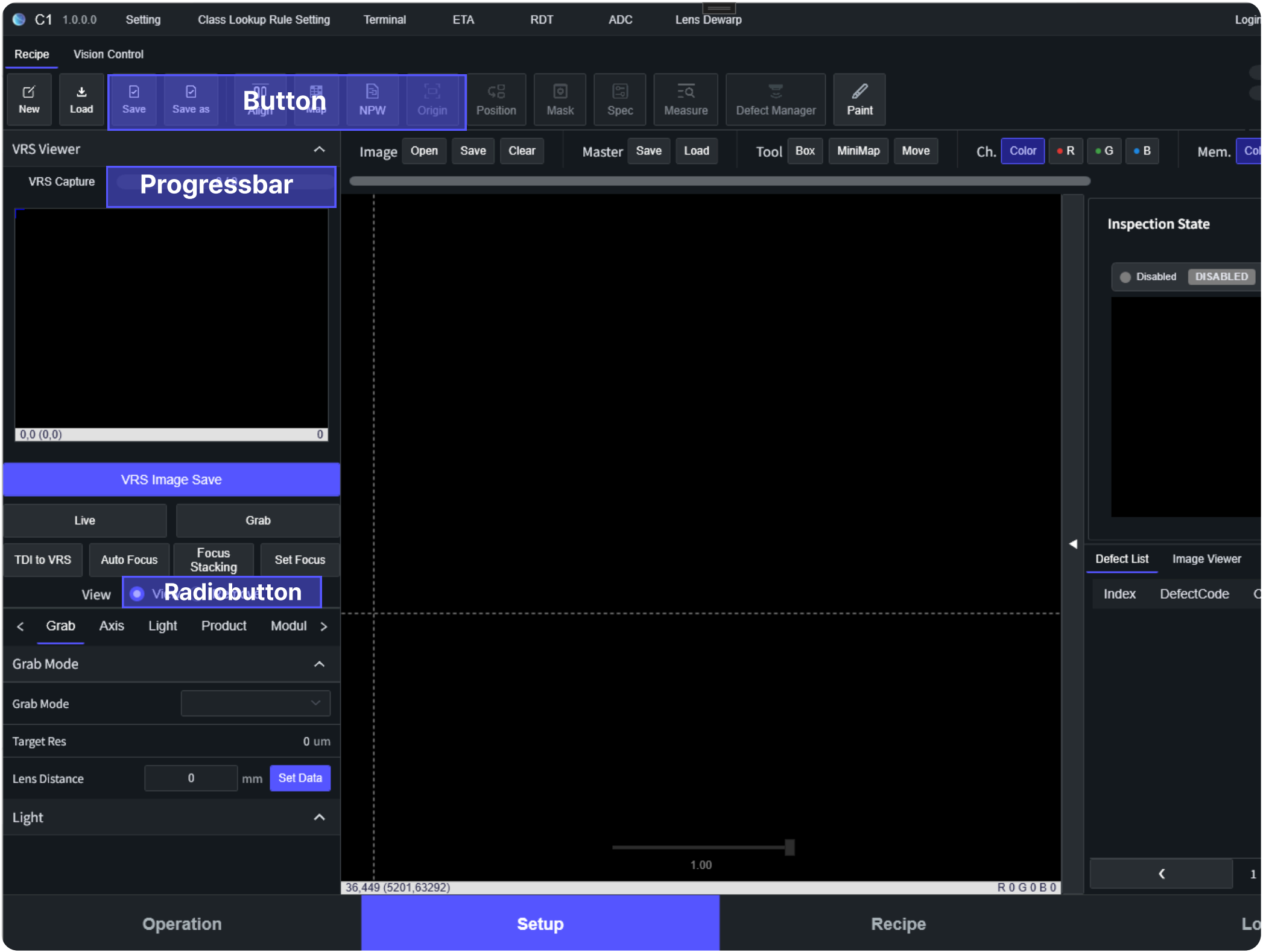
Button, TextBox, ComboBox와 같은 컴포넌트의 디자인과 상태를 미리 구현하여 개발자가 반복해서 작성하지 않고 프로젝트에 바로 적용할 수 있도록 제공합니다.

DOLPHIN UI 라이브러리의 역할

DOLPHIN Component



이와같이 Dolphin은 **ATI 설비 프로그램의 공통 UI의 모양과 사용 기준을** 제공하고, MVVM 패턴 적용 시 View 화면 Window UI에서 컴포넌트를 Style로 적용하여 프로그램 간 디자인과 사용자 경험을 일관되게 유지 가능합니다.



실제 C1 Oura에 현재 적용 중인 화면 (Dark Theme로 적용 중)

Dolphin에서 제공하는 Control과 Style

분류	개수
기본 입력 (Input)	11
리스트 & 데이터 (List / Data)	4
네비게이션 & 레이아웃 (Navigation / Layout)	8
피드백 & 상태 (Feedback / Status)	6
차트 & 시각화 (Chart / Visualization)	5
특수 (Specialized)	3
전체 (Total)	37

No.	분류	영문 분류	컴포넌트	파일
1	기본 입력	Input	Button	Button.xaml
2	기본 입력	Input	ButtonIcon	ButtonIcon.xaml
3	기본 입력	Input	ButtonGhost	ButtonGhost.xaml
4	기본 입력	Input	TextBox	TextBox.xaml
5	기본 입력	Input	PasswordBox	PasswordBox.xaml
6	기본 입력	Input	CheckBox	CheckBox.xaml
7	기본 입력	Input	CheckBoxButton	CheckBoxButton.xaml
8	기본 입력	Input	RadioButton	RadioButton.xaml
9	기본 입력	Input	ToggleButton	ToggleButton.xaml
10	기본 입력	Input	ToggleSwitch	ToggleSwitch.xaml
11	기본 입력	Input	ComboBox	ComboBox.xaml
12	리스트 & 데이터	List / Data	ListView / ListViewItem	ListView.xaml, ListViewItem.xaml
13	리스트 & 데이터	List / Data	ListBox / ListBoxItem	ListBox.xaml, ListBoxItem.xaml
14	리스트 & 데이터	List / Data	DataGrid	DataGrid.xaml
15	리스트 & 데이터	List / Data	TreeView / TreeViewItem	TreeView.xaml, TreeViewItem.xaml
16	네비게이션 & 레이아웃	Navigation / Layout	TabControl / TabItem	TabControl.xaml, TabItem.xaml
17	네비게이션 & 레이아웃	Navigation / Layout	Menu / MenuItem	Menu.xaml, MenuItem.xaml
18	네비게이션 & 레이아웃	Navigation / Layout	ContextMenu	ContextMenu.xaml
19	네비게이션 & 레이아웃	Navigation / Layout	BottomNav	BottomNav.xaml
20	네비게이션 & 레이아웃	Navigation / Layout	Accordion	Accordion.xaml

Dolphin에서 제공하는 Control과 Style

17	네비게이션 & 레이아웃	Navigation / Layout	Menu / MenuItem	Menu.xaml, MenuItem.xaml
18	네비게이션 & 레이아웃	Navigation / Layout	ContextMenu	ContextMenu.xaml
19	네비게이션 & 레이아웃	Navigation / Layout	BottomNav	BottomNav.xaml
20	네비게이션 & 레이아웃	Navigation / Layout	Accordion	Accordion.xaml
21	네비게이션 & 레이아웃	Navigation / Layout	GroupBox	GroupBox.xaml
22	네비게이션 & 레이아웃	Navigation / Layout	GridSplitter	GridSplitter.xaml
23	네비게이션 & 레이아웃	Navigation / Layout	ScrollViewer / ScrollBar	ScrollViewer.xaml, ScrollBar.xaml
24	피드백 & 상태	Feedback / Status	ProgressBar	ProgressBar.xaml
25	피드백 & 상태	Feedback / Status	CircularProgressBar	CircularProgressBar.xaml
26	피드백 & 상태	Feedback / Status	LoadingSpinner	LoadingSpinner.xaml
27	피드백 & 상태	Feedback / Status	Badge	Badge.xaml
28	피드백 & 상태	Feedback / Status	Popup	Popup.xaml
29	피드백 & 상태	Feedback / Status	MessageBox	MessageBox.xaml
30	차트 & 시각화	Chart / Visualization	BarChart	BarChart.xaml
31	차트 & 시각화	Chart / Visualization	LineChart	LineChart.xaml
32	차트 & 시각화	Chart / Visualization	DonutChart	DonutChart.xaml
33	차트 & 시각화	Chart / Visualization	RadialGauge	RadialGauge.xaml
34	차트 & 시각화	Chart / Visualization	ChartToolTip	ChartToolTip.xaml
35	특수	Specialized	FanControl	FanControl.xaml
36	특수	Specialized	DateRangeCalendar	DateRangeCalendar.xaml
37	특수	Specialized	TextBlock	TextBlock.xaml

개발자가 직접 구현해야 하는 기능 범위

즉 Dolphin UI 라이브러리는 공통 UI의 디자인과 기본 동작을 제공하지만,
각 프로그램에서 사용하는 업무 데이터와 기능 로직까지 구현하지는 않습니다.
개발자는 Dolphin 컴포넌트를 화면에 배치한 뒤, 프로젝트의 요구사항에 맞게 **ViewModel과 데이터를 연결**해야 합니다.

XML

```
<TextBox Text="{Binding RecipeName}" />
```

화면 데이터 구성

화면에 표시할 데이터를 ViewModel의 Property로 정의하고
컴포넌트에 Binding합니다.

XML

```
<Button Content="저장" Command="{Binding SaveCommand}" />
```

버튼 및 사용자 동작 연결

버튼 클릭, 항목 선택, 검색, 저장과 같은 동작을 Command로
구현합니다.

XML

```
<Button Content="실행" IsEnabled="{Binding CanRun}"  
Command="{Binding RunCommand}" />
```

업무 로직 구현

프로젝트마다 필요한 실제 기능은 개발자가 구현함

컴포넌트 상태 관리

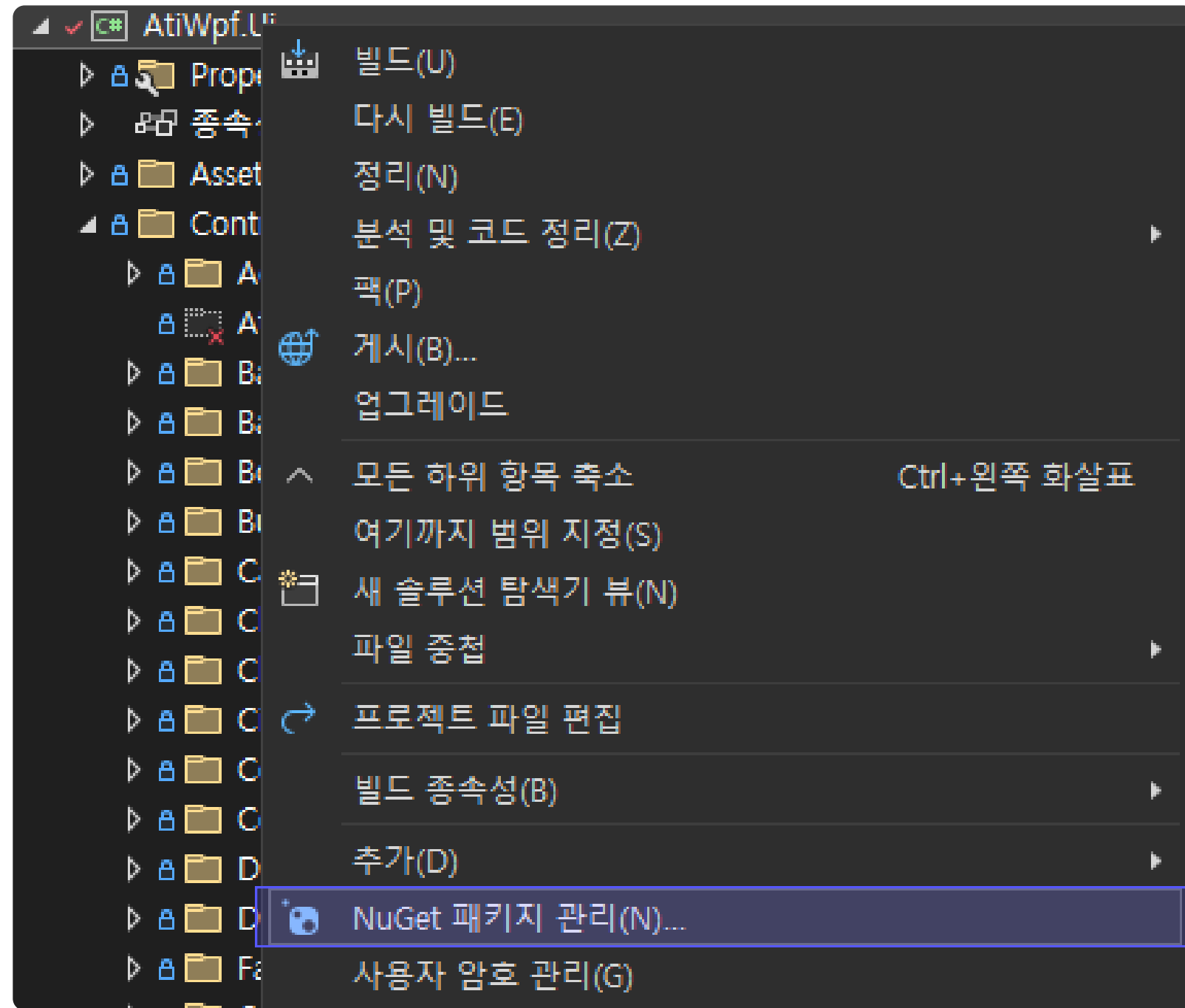
업무 상황에 따라 컴포넌트의 활성화 여부, 표시 여부, 선택 상태 등을 Binding으로 제어합니다.

설치 방법 및 업데이트 안내

Dolphin UI 라이브러리를 프로젝트에 설치하고 최신 버전으로 유지하는 방법을 안내합니다.



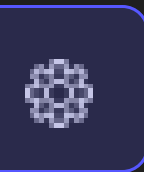
Dolphin UI 라이브러리 패키지 설치



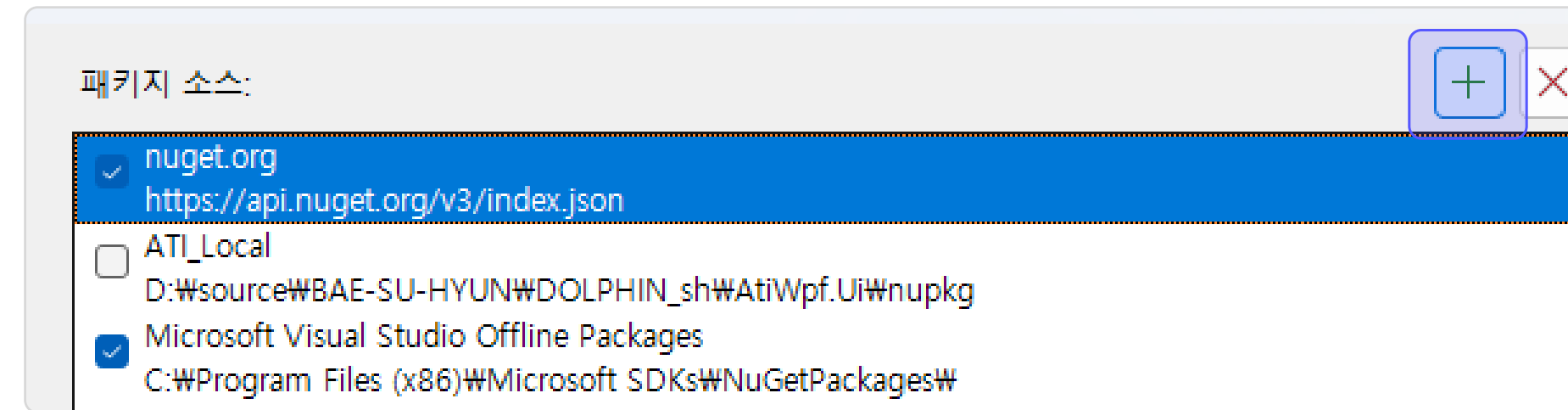
1. 프로젝트 우측 클릭 후 NuGet 패키지 관리(N)을 클릭

NuGet 패키지 관리자: AtiWpf.Ui

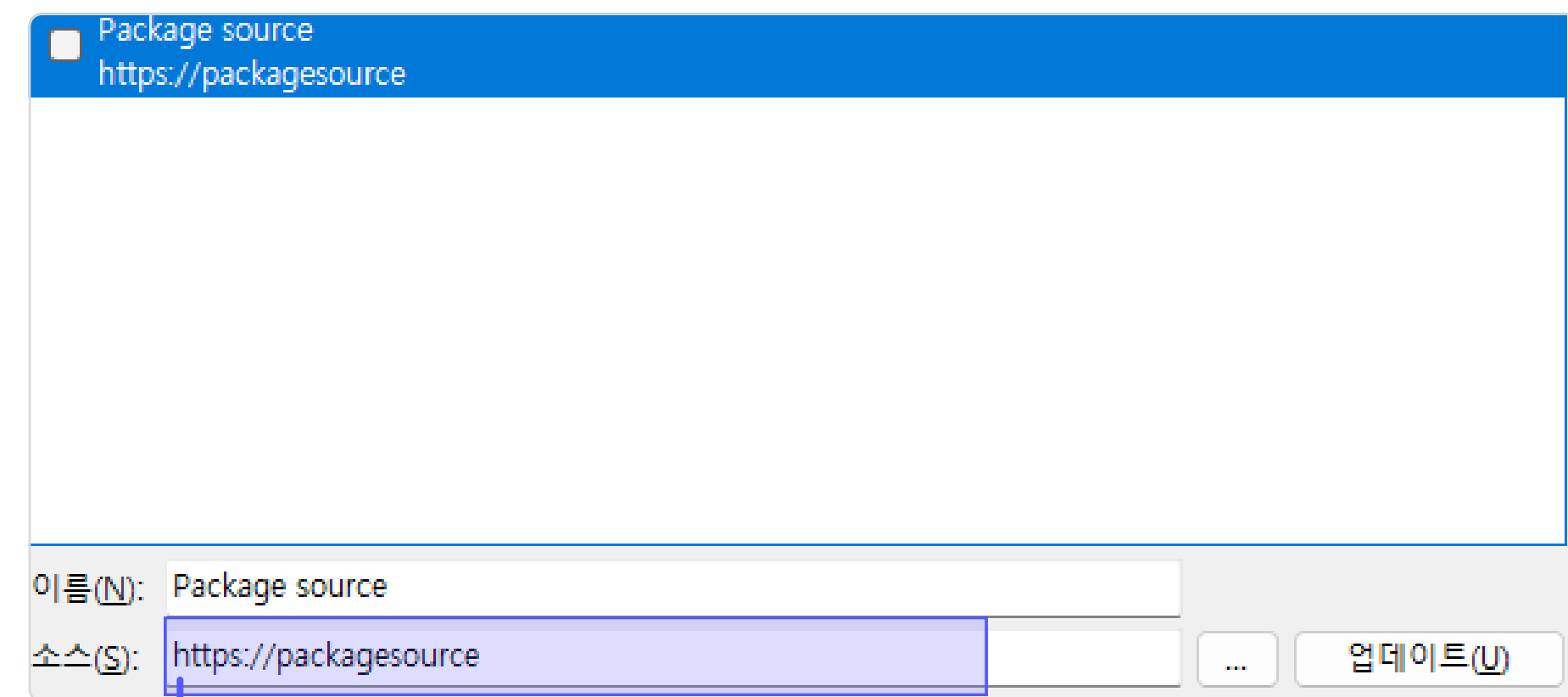
패키지 소스: nuget.org



2. 우측에 패키지 소스 설정 아이콘을 클릭



3. 패키지 소스 우측에 + 버튼을 클릭하여 패키지 소스 생성



4. 패키지 소스 링크 등록 후 업데이트 클릭

서버 주소

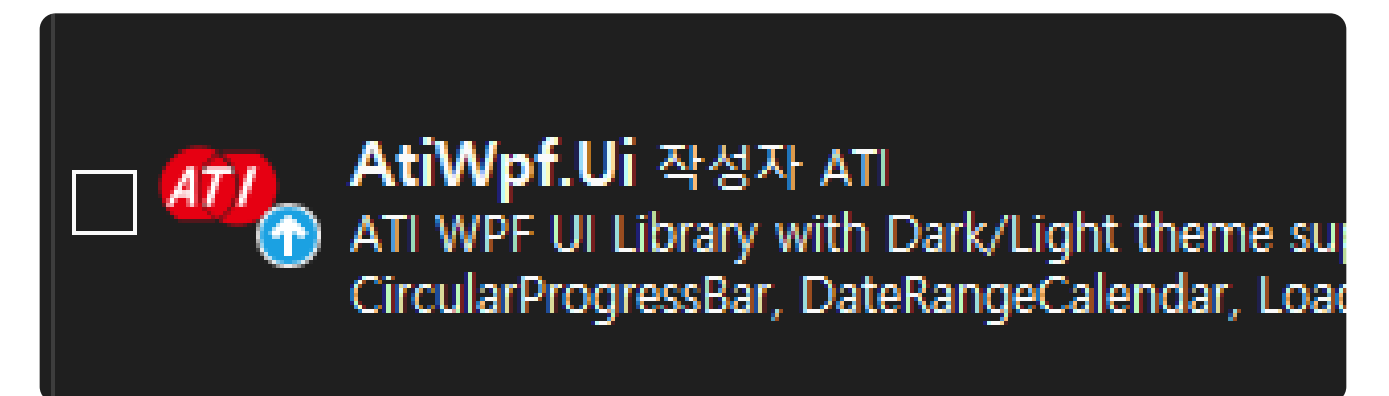
http://192.168.9.2:5000/v3/index.json

Dolphin UI 라이브러리 패키지 설치



5. 좌측에 찾아보기에 뜬 AtiWpf.Ui 패키지 파일 버전을 확인 후 우측에 설치하기 버튼을 눌러 설치 완료

6. ★ 만일 최신 버전으로 업데이트 해야하는 버전이 있을 경우 버전 업데이트 적용



업데이트 필요 시 업데이트 확인 필요

ResourceDictionary 및 Theme 적용

App.xaml

```
<Application x:Class="Oura.Cl.App"
    xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
    xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
    xmlns:local="clr-namespace:Oura.Cl"
    xmlns:converters="clr-namespace:Oura.Cl.Converters"
    Startup="Application_Startup">
    <Application.Resources>
        <ResourceDictionary>
            <ResourceDictionary.MergedDictionaries>
                <ResourceDictionary Source="/ColorPickerLib/component/Themes/Generic.xaml"/>
                <ResourceDictionary Source="pack://application:,,,/AtiWpf.Ui/component/Themes/ColorsDark.xaml"/>
                <ResourceDictionary Source="pack://application:,,,/AtiWpf.Ui/component/Themes/AtiWpf.Ui.xaml"/>
            </ResourceDictionary.MergedDictionaries>
        </ResourceDictionary>
    </Application.Resources>
</Application>
```

```
<ResourceDictionary.MergedDictionaries>
    <ResourceDictionary Source="/ColorPickerLib/component/Themes/Generic.xaml"/>
    <ResourceDictionary Source="pack://application:,,,/AtiWpf.Ui/component/Themes/ColorsDark.xaml"/>
    <ResourceDictionary Source="pack://application:,,,/AtiWpf.Ui/component/Themes/AtiWpf.Ui.xaml"/>
</ResourceDictionary.MergedDictionaries>
```

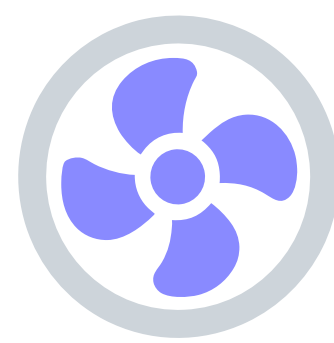
ColorsDark.xam → Theme 설정 (Light / Dark) 둘 중 선택 가능

AtiWpf.Ui.xaml → 컴포넌트 리소스 불러오기

App.xaml ResourceDictionary

- Dolphin UI 라이브러리의 컴포넌트와 Style을 사용하려면 ResourceDictionary에 Dolphin 리소스를 등록해야 합니다.
- 리소스가 정상적으로 연결되면 XAML에서 StaticResource를 통해 원하는 Style을 적용할 수 있으며, 동일한 Button이라도 Dolphin Style 적용 여부에 따라 디자인과 상태 표현이 달라집니다.
- Light·Dark Theme 역시 공통 리소스로 관리되므로, 테마에 맞는 Dictionary를 적용하면 색상과 배경, 텍스트 및 Control 상태가 전체에 동일하게 반영됩니다.

```
<UserControl x:Class="Oura.Cl.UI.Operation.OperationPage"
  xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
  xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
  xmlns:ati="clr-namespace:AtiWpf.Ui.Controls;assembly=AtiWpf.Ui"
  xmlns:controls="clr-namespace:Oura.Cl.UI.Operation.Controls"
  xmlns:d="http://schemas.microsoft.com/expression/blend/2008"
  xmlns:mc="http://schemas.openxmlformats.org/markup-compatibility/2006"
  mc:Ignorable="d"
  d:DesignWidth="1600" d:DesignHeight="900">
```



유형
임계값 초과
비율
40%
하드웨어
Motion

WPF에서 제공하지 않는 별도 UserControl을 호출하기 위해 등록 필수

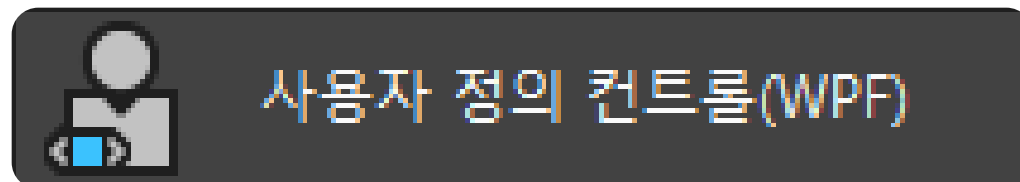
Window 또는 UserControl Namespace 등록

- Dolphin에서 제공하는 Custom Control을 XAML에서 사용하려면, 해당 Control이 포함된 네임스페이스를 Window 또는 UserControl의 최상단에 등록해야 합니다.
- xmlns:ati는 Dolphin UI 라이브러리에 포함된 Control을 현재 XAML 파일에서 사용할 수 있도록 연결하는 선언입니다.
- 등록된 dolphin 접두어를 통해 LoadingSpinner, CircularProgressBar, FanControl 등 Dolphin에서 별도로 구현한 Control을 호출할 수 있습니다.

View 화면 생성 안내

UserControl 또는 Window를 생성하고, ViewModel과 연결하여
기본 화면 구조를 구성하는 방법을 안내합니다.

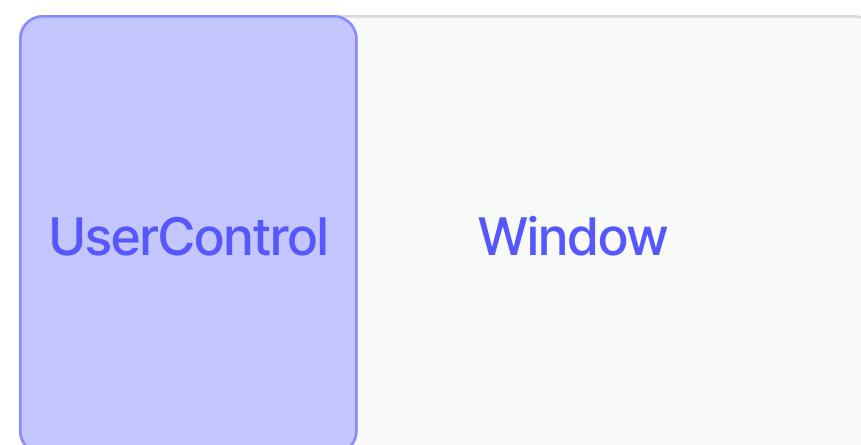




UserControl

프로그램 내부의 페이지나 콘텐츠 영역처럼
다른 화면 안에 배치되는 UI를 구성할 때 사용합니다.

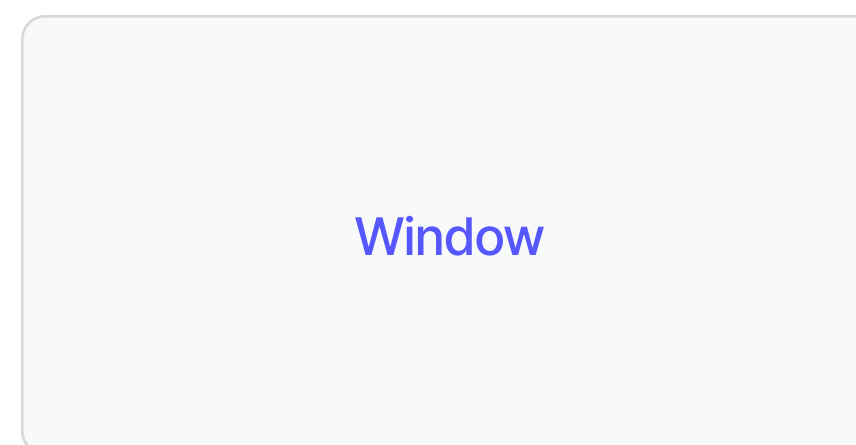
- 메뉴별 콘텐츠 화면
- 대시보드 및 설정 화면
- 반복해서 사용하는 화면 영역
- 다른 View 안에 포함되는 화면



Window

독립적으로 열리고 닫히는 별도의 화면을
구성할 때 사용합니다.

- 메인 프로그램 화면
- 상세 정보창
- 설정창
- 별도 팝업 또는 도구 화면



WPF의 MVVM 구조에서는
UserControl과 Window

모두 View 역할

WPF에서 UserControl과 Window는 모두 화면
UI를 구성하는 View 역할을 합니다.

UserControl은 다른 화면 내부에 배치되는 콘텐
츠 View로 사용하며, Window는 독립적으로 표시
되는 최상위 View로 사용합니다.

화면을 생성할 때는 UI를 담당하는 **View**와
화면에 표시할 데이터 및 동작을 담당하는 **ViewModel**을 구분하여 구성합니다.

Views

- └─ MonitoringView.xaml
- └─ MonitoringView.xaml.cs

ViewModels

- └─ MonitoringViewModel.cs

ViewModel은

View에 표시할 데이터와 사용자 동작을 관리합니다.

- 화면에 표시할 Property 정의
- 버튼 동작을 위한 Command 정의
- 선택, 활성화, 표시 여부 등의 상태 관리
- 데이터 조회·저장 등 기능 호출
- Property 변경 사항을 View에 전달

DataContext 연결

DataContext는

View에서 사용할 데이터와 기능의 기준이 되는 ViewModel을 연결하는 속성입니다.

연결 예시

```
public partial class MonitoringView : UserControl
{
    public MonitoringView()
    {
        InitializeComponent();
        DataContext = new MonitoringViewModel();
    }
}
```

ViewModel 데이터 Binding

```
<TextBlock Text="{Binding EquipmentName}" />

<Button
    Content="실행"
    Command="{Binding RunCommand}" />
```

코드에서 EquipmentName과 RunCommand는 DataContext로 연결된 MonitoringViewModel에 정의되어 있어야 합니다.

ViewModel.cs

```
public class MonitoringViewModel
{
    public string EquipmentName { get; set; } = "Equipment 01";

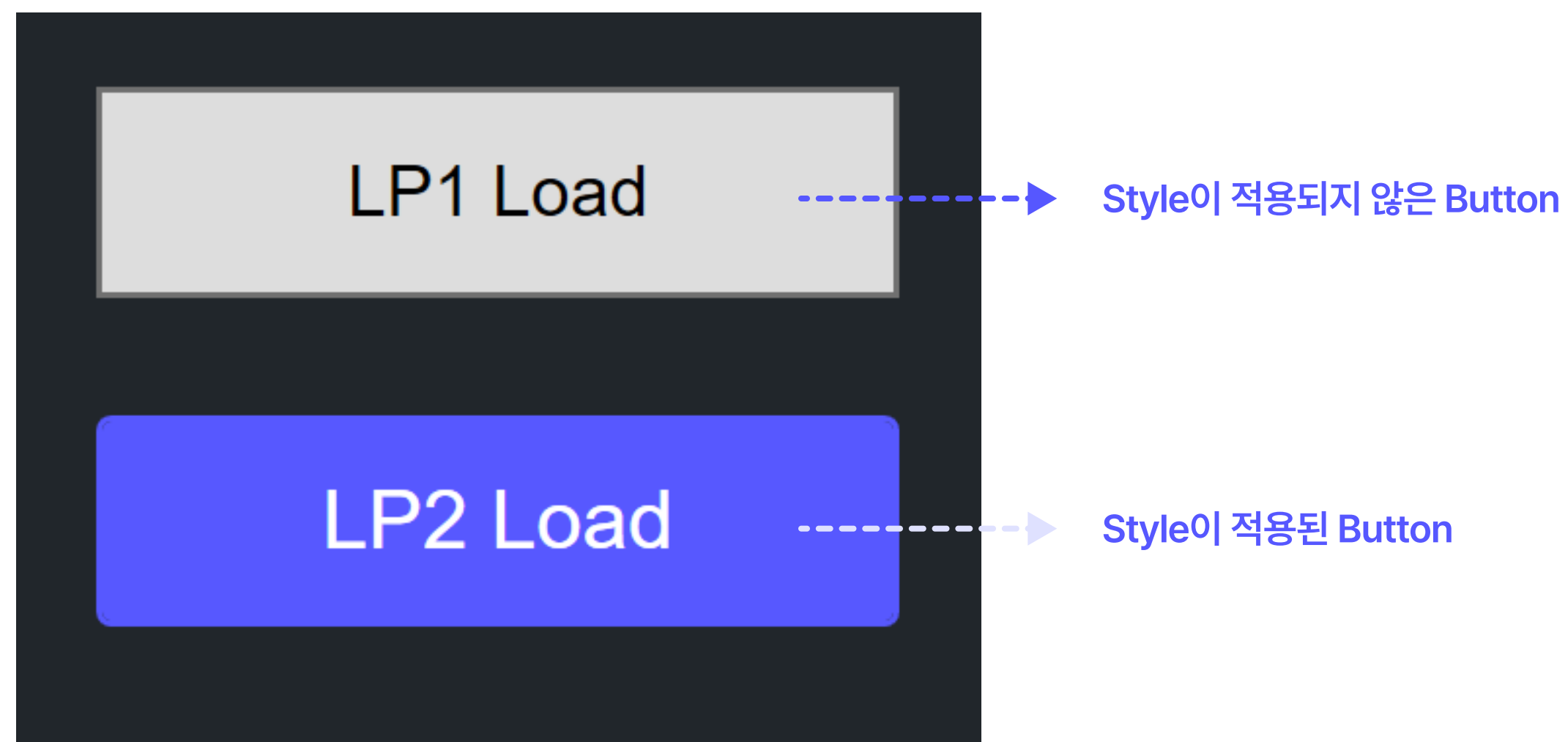
    public ICommand RunCommand { get; }
}
```

Dolphin 컴포넌트 적용

Dolphin에서 제공하는 컴포넌트와 Style을 화면에 적용하고,
데이터와 기능을 연결하는 방법을 안내합니다.



기본 WPF Control을 Dolphin Control로 구성



WPF 기본 컴포넌트 스타일

```
<ati:Button Content="LP1 Load" Height="36" Margin="0,20,0,20"  
            CornerRadius="2" HorizontalAlignment="Stretch"  
            Command="{Binding Loadport1LoadCommand}" />
```

```
<ati:Button Content="LP1 Load" Height="36" Margin="0,20,0,20"  
            CornerRadius="2" HorizontalAlignment="Stretch"  
            Style="{StaticResource button}"  
            Command="{Binding Loadport1LoadCommand}" />  
<ati:Button Content="LP2 Load" Height="36" Margin="0,20,0,20"  
            CornerRadius="2" HorizontalAlignment="Stretch"  
            Style="{StaticResource button}"  
            Command="{Binding Loadport2LoadCommand}" />  
<Border Height="1" Background="{StaticResource separator}" />  
<ati:Button Style="{StaticResource button}" Content="Home" Height="36" Margin="0,20,0,20"  
            CornerRadius="2" HorizontalAlignment="Stretch"  
            Command="{Binding HomeCommand}" />  
<ati:Button Style="{StaticResource button}" Content="Load" Height="36" Margin="0,20,0,20"  
            CornerRadius="2" HorizontalAlignment="Stretch"  
            Command="{Binding LoadCommand}" />
```

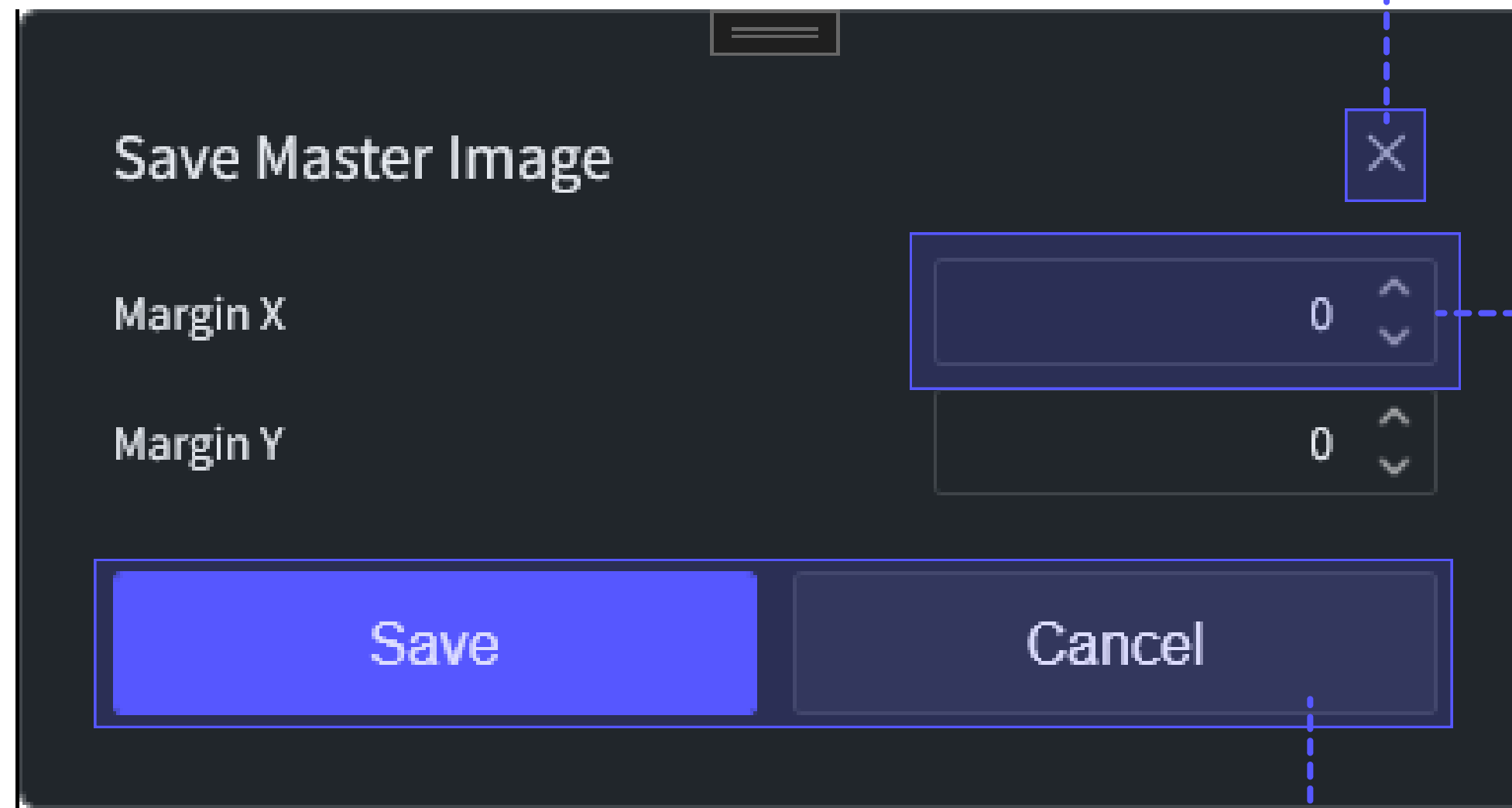
- BoxViewer_toolbar_button
- ButtonCommonBase
- ButtonDefaultBase
- ButtonIconCommonBase
- ButtonIconDefaultBase
- ButtonIconLarge
- ButtonIconLargeSolid
- ButtonIconMedium
- ButtonIconMediumSolid

Button 컴포넌트에
해당하는 Style 리소스 키

별도의 Style을 지정하지 않으면 WPF에서 기본으로 제공하는 디자인이 적용됩니다.

Dolphin UI 라이브러리 설치 후에는 Style 속성에 {StaticResource 스타일 리소스 키} 또는 {DynamicResource 스타일 리소스 키}를 입력하여 원하는 Dolphin 버튼 Style을 적용할 수 있습니다.

Button, TextBox 등 적용 예시



예시: SaveMasterImagePopUp

```
<Button DockPanel.Dock="Right"
        Style="{StaticResource GhostIconButton}"
        HorizontalAlignment="Right"
        Width="28" Height="28"
        Command="{Binding CmdCancel}">
    <Image Source="/Image/Icons/close_default_dark.png"
        Width="18"
        Height="18"
        Stretch="Uniform"/>
</Button>
```

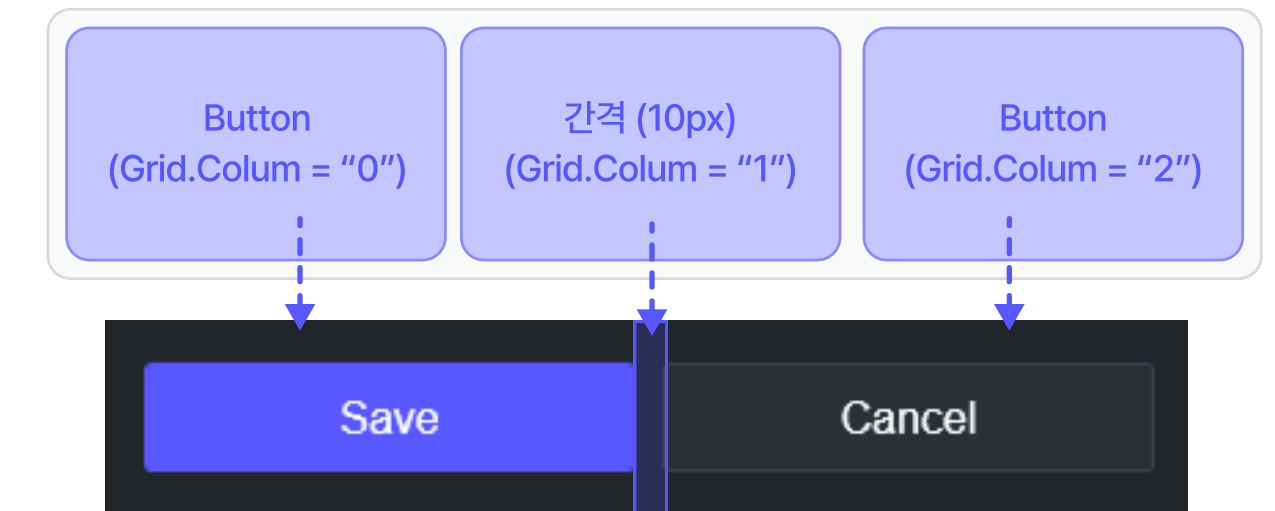
GhostIconButton의 경우 형태가 없음
Button 내부에 들어갈 image가 필수로
들어가기 때문에 적용 필요

```
<ati:TextBox DockPanel.Dock="Right" Width="140" HorizontalAlignment="Right"
        Style="{StaticResource TextBoxSmallNumericUpDown}"
        IsNumericUpDownEnabled="True"
        Minimum="-999999" Maximum="999999" Step="1"
        Value="{Binding MarginX, UpdateSourceTrigger=PropertyChanged}"
        Validation.ErrorTemplate="{x:Null}"
        HorizontalContentAlignment="Right"/>
```

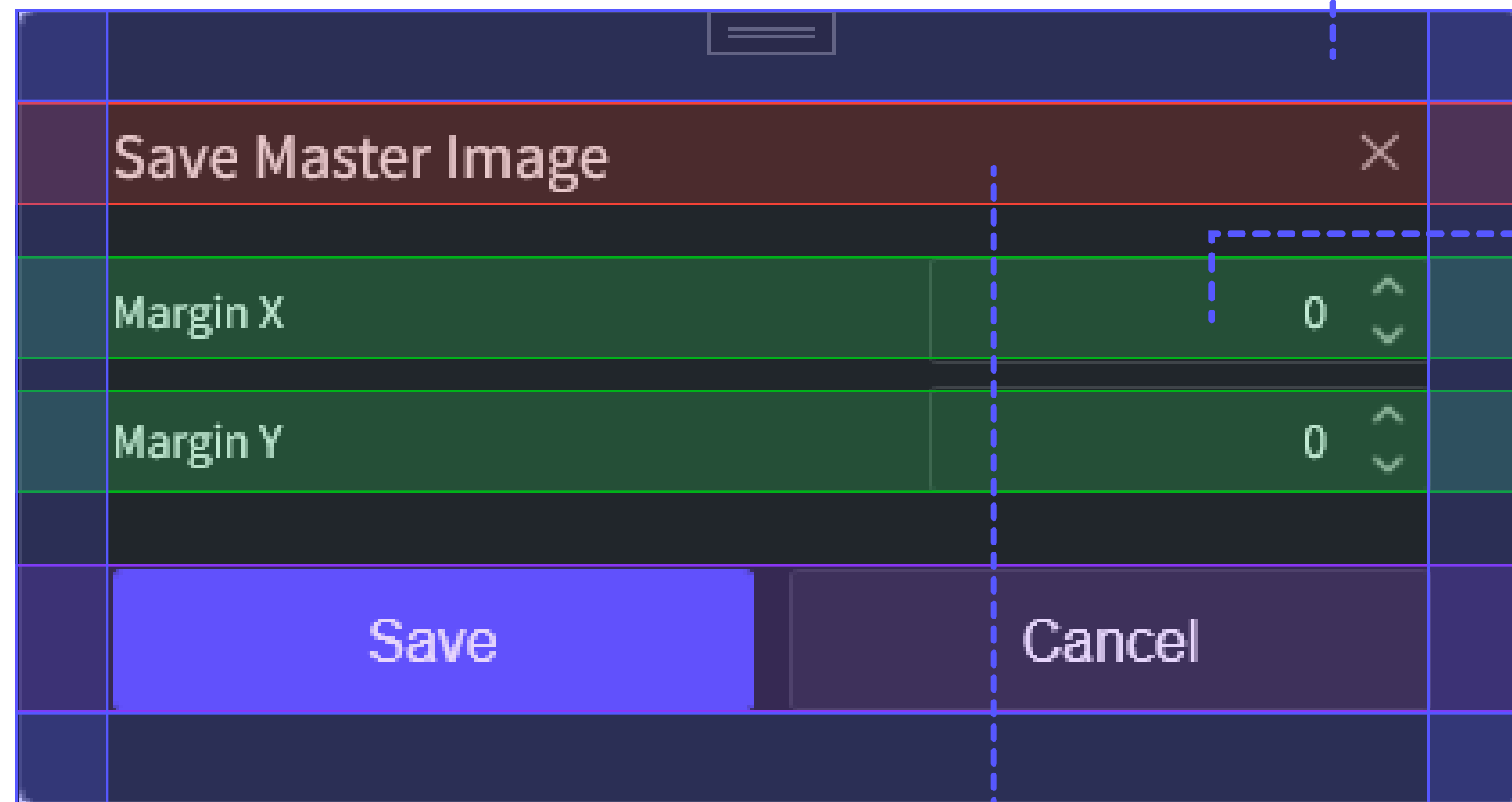
TextboxSmallNumericUpDown
Style 키 리소스 적용

```
<Grid Margin="0,15,0,0" Height="40">
    <Grid.ColumnDefinitions>
        <ColumnDefinition Width="*" />
        <ColumnDefinition Width="10" />
        <ColumnDefinition Width="*" />
    </Grid.ColumnDefinitions>
    <ati:Button Grid.Column="0"
        Style="{StaticResource ButtonLargeSolid}"
        CornerRadius="2"
        HorizontalAlignment="Stretch"
        Content="Save"
        Command="{Binding CmdSave}" />
    <ati:Button Grid.Column="2"
        Style="{StaticResource ButtonLarge}"
        CornerRadius="2"
        HorizontalAlignment="Stretch"
        Content="Cancel"
        Command="{Binding CmdCancel}" />
</Grid>
```

두 개의 Button을 가로로 나란히 배치하기 위해
부모 Grid에 Grid.ColumnDefinitions를 정의합니다.
버튼 영역은 동일한 너비의 두 열로 구성하고,
가운데에는 10의 간격을 둡니다. 이후 각 Button에
Grid.Column 값을 지정하여 원하는 열에 배치합니다.



기본 레이아웃 구성 예시



25

```
<Border Background="{DynamicResource ContentBackgroundPrimary}"
        BorderBrush="{DynamicResource ContentBorderPrimary}"
        BorderThickness="1"
        CornerRadius="4"
        Padding="25"
        MouseLeftButtonDown="Border_MouseLeftButtonDown">
```

① 전체 컨테이너 구성

팝업의 최상위 영역은 Border로 구성하며, Background와 BorderBrush에 Dolphin Theme Resource를 적용합니다. Padding="25"를 지정하여 외곽선과 내부 콘텐츠 사이의 여백을 일정하게 유지합니다.

```
<!-- Margin X -->
<DockPanel Margin="0,0,0,6">
  <TextBlock Text="Margin X"
    Style="{StaticResource ContentSmall}"
    Foreground="{DynamicResource ContentText}"
    VerticalAlignment="Center"/>
  <ati:TextBox DockPanel.Dock="Right" Width="140" HorizontalAlignment="Right"
    Style="{StaticResource TextBoxSmallNumericUpDown}"
    IsNumericUpDownEnabled="True"
    Minimum="-999999" Maximum="999999" Step="1"
    Value="{Binding MarginX, UpdateSourceTrigger=PropertyChanged}"
    Validation.ErrorTemplate="{x:Null}"
    HorizontalContentAlignment="Right"/>
</DockPanel>
```

② 입력 영역 구성

입력 항목은 DockPanel 또는 Grid를 사용해 라벨과 입력 Control을 한 행에 배치합니다. TextBlock과 NumericUpDown에 Dolphin Style을 적용하고, 항목 사이의 간격은 Margin으로 조정합니다.

```
<!-- 헤더: "Save Master Image" + 닫기 -->
<DockPanel Margin="0,0,0,15">
  <TextBlock Text="Save Master Image"
    Style="{StaticResource ContentLarge}"
    FontWeight="Medium"
    VerticalAlignment="Center"/>
  <Button DockPanel.Dock="Right"
    Style="{StaticResource GhostIconButton}"
    HorizontalAlignment="Right"
    Width="28" Height="28"
    Command="{Binding CmdCancel}">
    <Image Source="/image/icons/close_default_dark.png"
      Width="18"
      Height="18"
      Stretch="Uniform"/>
  </Button>
</DockPanel>
```

③ 제목 및 버튼 영역 구성

상단에는 제목과 닫기 버튼을 좌우로 배치하며, 닫기 버튼에는 GhostIconButton Style과 이미지 리소스를 적용합니다. 하단의 Save와 Cancel 버튼은 동작의 중요도에 따라 Solid와 보조 Button Style로 구분합니다.